

OS Lab

Skill-8

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            i++;

            char variable[100];
            int k = 0;

            /* Detect ${VARIABLE} format */
            if (input[i] == '{')
            {
                i++;

                while (input[i] != '\0' && input[i] != '}' && k < 99)
                {
                    variable[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;
            }
        }
        output[j++] = input[i++];
    }
    output[j] = '\0';
}
```

```

    }
    else
    {
        /* Detect $VARIABLE format */
        while (input[i] != '\0' &&
            (isalnum(input[i]) || input[i] == '_') &&
            k < 99)
        {
            variable[k++] = input[i++];
        }
    }

    variable[k] = '\0';

    if (k == 0)
    {
        output[j++] = '$';
        continue;
    }

    char *value = getenv(variable);

    if (value != NULL)
    {
        strcpy(&output[j], value);
        j += strlen(value);
    }
    else
    {
        printf("Undefined variable: %s\n", variable);
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];

```

```

char output[MAX_INPUT * 2];

printf("Variable Expansion Shell\n");
printf("Type 'exit' to quit.\n");

while (1)
{
    printf("var-shell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

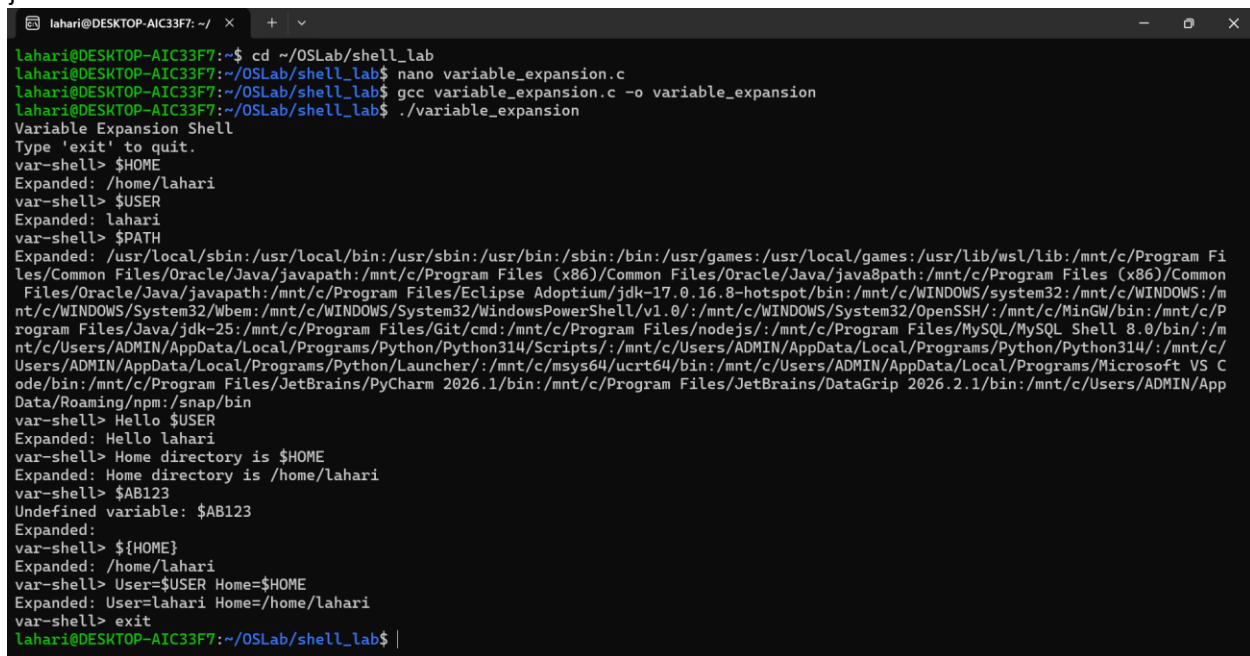
    if (strcmp(input, "exit") == 0)
        break;

    expand_variables(input, output);

    printf("Expanded: %s\n", output);
}

return 0;
}

```



```

lahari@DESKTOP-AIC33F7: ~/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano variable_expansion.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc variable_expansion.c -o variable_expansion
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./variable_expansion
Variable Expansion Shell
Type 'exit' to quit.
var-shell> $HOME
Expanded: /home/lahari
var-shell> $USER
Expanded: lahari
var-shell> $PATH
Expanded: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files/Eclipse Adoptium/jdk-17.0.16-hotspot/bin:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/MinGW/bin:/mnt/c/Program Files/Java/jdk-25:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/ADMIN/AppData/Local/Programs/Python/Python314/Scripts:/mnt/c/Users/ADMIN/AppData/Local/Programs/Python/Python314:/mnt/c/Users/ADMIN/AppData/Local/Programs/Python/Launcher:/mnt/c/msys64/ucrt64/bin:/mnt/c/Users/ADMIN/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Program Files/JetBrains/DataGrip 2026.2.1/bin:/mnt/c/Users/ADMIN/AppData/Roaming/npm:/snap/bin
var-shell> Hello $USER
Expanded: Hello lahari
var-shell> Home directory is $HOME
Expanded: Home directory is /home/lahari
var-shell> $AB123
Undefined variable: $AB123
Expanded:
var-shell> ${HOME}
Expanded: /home/lahari
var-shell> User=$USER Home=$HOME
Expanded: User=lahari Home=/home/lahari
var-shell> exit
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$

```

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

void builtin_pwd(char **args)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args[1]) != 0)
        perror("cd");
}

void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");
}

```

```

}

void builtin_help(char **args)
{
    printf("\nBuilt-in commands:\n");
    printf(" cd <directory> - Change directory\n");
    printf(" pwd          - Show current directory\n");
    printf(" echo <text>    - Display text\n");
    printf(" help          - Show available commands\n");
    printf(" exit          - Exit shell\n\n");
}

void builtin_exit(char **args)
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Dispatch table */

struct builtin_command
{
    char *name;
    void (*function)(char **args);
};

struct builtin_command builtins[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(builtins) / sizeof(builtins[0]);

int execute_builtin(char **args)
{
    if (args[0] == NULL)
        return 0;

    for (int i = 0; i < builtin_count; i++)

```

```

    {
        if (strcmp(args[0], builtins[i].name) == 0)
        {
            builtins[i].function(args);
            return 1;
        }
    }

    return 0;
}

int parse_command(char *input, char **args)
{
    int count = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;

    return count;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Built-in Command Shell\n");
    printf("Type 'help' for available commands.\n\n");

    while (1)
    {
        printf("shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }
}

```

```

    if (strlen(input) == 0)
        continue;

    parse_command(input, args);

    if (execute_builtin(args))
        continue;

    printf("Invalid command: %s\n", args[0]);
}

return 0;
}

```

```

lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ cd ~/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano builtin_shell.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc builtin_shell.c -o builtin_shell
builtin_shell.c:7: warning: "MAX_INPUT" redefined
    7 | #define MAX_INPUT 1024
      |
In file included from /usr/include/x86_64-linux-gnu/bits/local_lim.h:38,
                 from /usr/include/x86_64-linux-gnu/bits/posix1_lim.h:161,
                 from /usr/include/limits.h:195,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:205,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/syslimits.h:7,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:34,
                 from builtin_shell.c:5:
/usr/include/linux/limits.h:11: note: this is the location of the previous definition
   11 | #define MAX_INPUT      255      /* size of the type-ahead buffer */
      |
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./builtin_shell
Built-in Command Shell
Type 'help' for available commands.

shell> help

Built-in commands:
cd <directory> - Change directory
pwd           - Show current directory
echo <text>    - Display text
help          - Show available commands
exit          - Exit shell

shell> pwd
/home/lahari/OSLab/shell_lab
shell> cd . .
shell> cd ..

```

```
11 | #define MAX_INPUT      255    /* size of the type-ahead buffer */
|
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./builtin_shell
Built-in Command Shell
Type 'help' for available commands.

shell> help

Built-in commands:
  cd <directory>  - Change directory
  pwd             - Show current directory
  echo <text>     - Display text
  help           - Show available commands
  exit           - Exit shell

shell> pwd
/home/lahari/OSLab/shell_lab
shell> cd . .
shell> cd ..
shell> pwd
/home/lahari/OSLab
shell> cd shell_lab
shell> pwd
/home/lahari/OSLab/shell_lab
shell> echo Hello
Hello
shell> echo Hello World
Hello World
shell> abc123
Invalid command: abc123
shell> exit
Exiting shell...
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ |
```